CSC10014 · PRACTICAL TRACK · 2026

Lab 5

Your First Database

Tables, keys and SQL from zero — then the assistant remembers feedback, safely

Practical Track · Week 5 · 3 h · No prior database experience needed

Faculty of Information Technology · University of Science, VNU-HCM

Instructor: Dr. LE Thanh Tung

Contents

About This Lab

Lab 5 — Your First Database. Practical track, Week 5. Duration: 3 hours in class, plus a short homework.

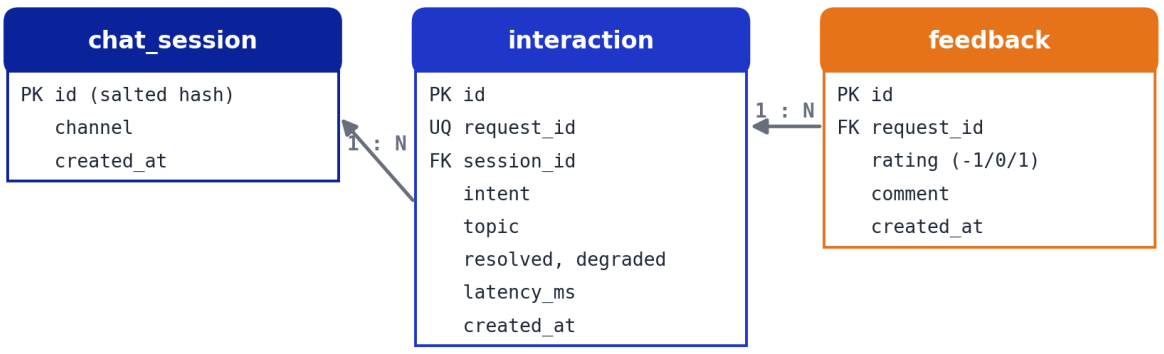
This lab assumes you have **never used a database**. You know Python lists, dicts, classes and files — we start from there, one small step at a time. Most of the time you will practise on a tiny, friendly *school* database before touching the assistant.

What you already know → what is new today

You already know	Today you learn	The link
a list of objects `students = [Student(...), ...]`	a **table** with rows	a table is like a list of records, stored on disk
a `dict` key must be unique	a **primary key**	the unique id of a row
an object that stores the id of another object	a **foreign key**	a column pointing to a row in another table
a `for` loop with `if` to filter a list	**SQL** `SELECT ... WHERE ...`	you describe *what* you want, the database finds it
grouping and counting with a dict	`GROUP BY` and `COUNT`	the database counts for you
a class with typed fields	a **model** (SQLModel)	one class = one table

Goal

Learn to store, find and count data with SQL, then make the assistant **remember** feedback and basic information about each question — without storing anything personal.



No raw messages, no names: only metadata, a hashed session id and feedback.

Figure 0.1 — By the end of today, the assistant uses these three tables. Do not worry about the details yet.

What you hand in

Level	Task	Where
Must	SQL practice Q1–Q10 on `school.db` (checked by `check.py`)	`practice/my_answers.sql`

Level	Task	Where
Must	Python practice D1–D3 (`school_repo.py`) — tests green	`practice/`
Must	assistant tasks R1–R2 (`starter/`) — tests green	PR
Should	SQL practice Q11–Q15 (joins)	`practice/my_answers.sql`
Should	three queries on the assistant data (`sql/my_queries.sql`)	PR
Bonus	design worksheet: what we should *not* store, and why	`docs/lab5/`

Timeline

Time	Activity
0:00–0:20	Part A: why databases?
0:20–0:40	Part B: tables, rows, keys — on paper
0:40–1:40	Part C: SQL step by step on `school.db` (Q1–Q15)
1:40–1:50	break
1:50–2:20	Part D: SQL from Python (D1–D3)
2:20–2:50	Part E: the assistant remembers (R1–R2)
2:50–3:00	Part F preview; submit

Before Class

1. Lab 4 works: the API starts and its tests pass.
2. Unzip the Lab 5 materials. You will use `lab05/practice/` and `lab05/starter/`.
3. Install **DB Browser for SQLite** (free, sqlitebrowser.org) — a window where you can look at tables and run SQL. Or install the *SQLite Viewer* extension in VS Code.
4. Optional (30 minutes): SQLBolt lessons 1–4 (sqlbolt.com).

Set up the starter environment once:

```
$ cd lab05/starter
$ python -m venv .venv
$ source .venv/bin/activate            # Windows: .venv\Scripts\activate
(.venv) $ pip install -r requirements.txt
(.venv) $ pip install -e .
(.venv) $ cp .env.example .env
```

Part A — Why Databases?

A.1 Where does our data live today?

In Lab 4, feedback lived in a Python list:

```
feedback_store: list[FeedbackOut] = []
```

Try it: start the API, send feedback, stop the server (Ctrl+C), start it again, and list the feedback. **It is gone.** A list lives in memory; memory is cleared when the program stops.

A.2 Why not just a file?

We could write the list to a JSON file. That works for a toy, but:

Question	With a file	With a database
Does data survive a restart?	yes	yes
Two users save at the same moment?	one may overwrite the other	the database handles it safely
"How many thumbs-down per topic?"	load everything, write loops	one short query
Find one record among a million?	read the whole file	an index finds it fast
Block a rating of 7?	hope your code checks	the database refuses it

A **database** is a program that stores data on disk and lets you add, find, change and count it safely. Today we use **SQLite**: the whole database is one file, and Python can talk to it without installing anything else.

Part B — Tables, Rows, Keys (On Paper First)

B.1 A table is like a spreadsheet

Here is the `students` table of our practice database:

id	name	year	program
1	An	1	CS
2	Binh	2	SE
3	Chi	2	CS
4	Dung	3	AI

- Each **row** is one student (one record).
- Each **column** has a name and a **type**: `id` and `year` are integers, `name` and `program` are text.
- Every row must have the same columns — like objects of the same class.

B.2 The primary key

`id` is the **primary key**: every student has a different id, and it never changes. Like a key in a Python dict, it lets us point to exactly one row. Names are not good keys — two students can be called "An".

B.3 Relationships and foreign keys

Students take courses. Where do we store "An takes CSC10001 and got 8.5"?

- Not in `students` — a student takes many courses.
- Not in `courses` — a course has many students.

We add a third table, `enrollments`, with one row per (student, course):

student_id	course_code	grade
1	CSC10001	8.5
1	MTH00003	6.0
2	CSC10014	*(empty)*

`student_id` is a **foreign key**: it contains the primary key of a row in `students`. It works like an object that stores a reference to another object in OOP. An empty cell is called **NULL** in databases — like `None` in Python. Here it means "not graded yet".

B.4 The three practice tables

Table	Columns	Key
`students`	id, name, year, program	`id`
`courses`	code, title, credits	`code`
`enrollments`	student_id → students, course_code → courses, grade	(student_id, course_code) together

CHECK YOURSELF

1) Why is `name` a bad primary key?

2) Where do we store a grade, and why not in `students`?

3) What does NULL mean in the `grade` column?

Part C — SQL, Step by Step

SQL (Structured Query Language) is how we talk to a database. You describe *what* you want; the database decides *how* to find it.

C.1 Create the practice database

```
(.venv) $ cd lab05/practice
(.venv) $ python make_school_db.py
students     8 rows
courses      5 rows
enrollments  17 rows
```

This created the file `school.db`. Open it in **DB Browser for SQLite** (*Open Database*) and look at each table in the *Browse Data* tab. Then use the *Execute SQL* tab for the exercises below.

C.2 How you practise and check

Write each answer under its marker in `my_answers.sql`:

```
-- Q1  (all columns of all students)
SELECT * FROM students;
```

Then check all your answers at once:

```
(.venv) $ python check.py
Q1   OK
Q2   ..  not answered yet
Q3   NO  expected [['An'], ['Chi'], ['Hoa']], got [['Binh'], ['Khoa']]
...
Score: 1/15
```

`check.py` runs your queries on `school.db` and compares the results with the expected ones. Try as many times as you like.

C.3 SELECT and FROM — show me data

```
SELECT * FROM students;           -- * means "all columns"
SELECT name, year FROM students;  -- only these columns
```

> **EXERCISE**
> **Q1** all columns of all students. **Q2** only `name` and `year` of every student.

C.4 WHERE — only some rows

```
SELECT name FROM students WHERE program = 'CS';
```

```
SELECT name, year FROM students WHERE year >= 3;
SELECT name FROM students WHERE program = 'CS' AND year = 2;
```

SQL	Python
`=`	`==`
`<>` or `!=`	`!=`
`AND`, `OR`, `NOT`	`and`, `or`, `not`
`'CS'` (single quotes)	`"CS"`

EXERCISE

Q3 names of CS students. **Q4** name and year of students in year 3 or higher.

C.5 ORDER BY — sorting

```
SELECT name FROM students ORDER BY name;        -- A to Z
SELECT name FROM students ORDER BY name DESC;   -- Z to A
```

EXERCISE

Q5 student names from Z to A. **Q6** title and credits of 4-credit courses, sorted by title.

C.6 COUNT, AVG and GROUP BY — let the database count

```
SELECT COUNT(*) FROM students;                          -- how many rows?
SELECT AVG(grade) FROM enrollments;                     -- average (NULLs are skipped)
SELECT ROUND(AVG(grade), 2) FROM enrollments;           -- rounded to 2 decimals
SELECT program, COUNT(*) FROM students GROUP BY program;
```

`GROUP BY program` makes one group per program and counts each — the same as this Python:

```
counts = {}
for s in students:
    counts[s.program] = counts.get(s.program, 0) + 1
```

EXERCISE

Q7 how many students? **Q8** number of students per program, sorted by program. **Q9** average grade in `CSC10004`, rounded to 2 decimals.

C.7 NULL — the empty value

`NULL` is not a value you can compare with `=`. Use `IS NULL` / `IS NOT NULL`:

```
SELECT student_id, course_code FROM enrollments WHERE grade IS NULL;
```

EXERCISE

Q10 `student_id` and `course_code` of enrollments without a grade, sorted by `student_id`.

TIP

Stop here and run `python check.py`. Q1–Q10 are the **must** part. Q11–Q15 are the **should** part.

C.8 JOIN — combine two tables

`enrollments` has only ids. To see *names*, we join it with `students`:

```
SELECT s.name, e.grade
FROM enrollments e
JOIN students s ON s.id = e.student_id
WHERE e.course_code = 'CSC10014';
```

Read it like this: *"for every enrollment `e`, find the student `s` whose `id` equals `e.student_id`, and show their name and the grade."* `e` and `s` are short nicknames (aliases) for the tables.

It is the database version of this Python:

```
by_id = {s.id: s for s in students}
for e in enrollments:
    if e.course_code == "CSC10014":
        print(by_id[e.student_id].name, e.grade)
```

EXERCISE

Q11 name and grade of *graded* students in CSC10014, by name. **Q12** each course title with the number of enrolled students, most first, then by title. **Q13** names of students with a grade below 5.

C.9 Two joins, and LEFT JOIN

```
-- total credits per student: enrollments -> students and enrollments -> courses
SELECT s.name, SUM(c.credits) AS credits
FROM enrollments e
JOIN students s ON s.id = e.student_id
JOIN courses  c ON c.code = e.course_code
GROUP BY s.name
ORDER BY credits DESC, s.name;
```

A normal `JOIN` keeps only rows that match on both sides. A `LEFT JOIN` keeps **all** rows of the left table, and fills the missing side with NULL — useful to find what is *missing*:

```
-- students NOT enrolled in CSC10014
SELECT s.name
FROM students s
LEFT JOIN enrollments e ON e.student_id = s.id AND e.course_code = 'CSC10014'
```

```
WHERE e.student_id IS NULL
ORDER BY s.name;
```

EXERCISE

Q14 each student's name and total credits enrolled, most first, then by name. **Q15** names of students not enrolled in CSC10014.

C.10 Changing data: INSERT, UPDATE, DELETE

Try these in DB Browser (then click *Write Changes*), or on a copy — you can always recreate `school.db` with `make_school_db.py`:

```
INSERT INTO students (name, year, program) VALUES ('Minh', 1, 'CS');
UPDATE enrollments SET grade = 7.0 WHERE student_id = 2 AND course_code = 'CSC10014';
DELETE FROM enrollments WHERE student_id = 5;
```

WARNING

`UPDATE` or `DELETE` **without** `WHERE` changes every row in the table. Always write the `WHERE` first.

C.11 SQL summary card

Want to…	SQL
see columns	`SELECT a, b FROM t`
filter rows	`WHERE a = 1 AND b > 2`
sort	`ORDER BY a DESC`
count / average	`COUNT(*)`, `AVG(x)`, `SUM(x)`, `ROUND(x, 2)`
per group	`GROUP BY a`
empty values	`IS NULL`, `IS NOT NULL`
combine tables	`JOIN t2 ON t2.id = t1.t2_id`
keep unmatched rows	`LEFT JOIN`
add / change / remove	`INSERT INTO`, `UPDATE … SET … WHERE`, `DELETE FROM … WHERE`

Part D — SQL from Python

Programs send SQL to the database through a library. Python includes `sqlite3`.

D.1 The pattern

```
import sqlite3

con = sqlite3.connect("school.db")                              # open (creates the file if
missing)
rows = con.execute("SELECT name FROM students WHERE program = ?", ("CS",)).fetchall()
print(rows)                                                      # [('An',), ('Chi',), ('Hoa',)]
con.close()
```

- `execute(sql, values)` runs one statement.
- `?` is a **placeholder**; the value comes from the tuple. Note the comma in `("CS",)` — a tuple with one item.
- `fetchall()` returns a list of tuples, one per row.
- After `INSERT`, `UPDATE` or `DELETE`, call `con.commit()` to save.

D.2 Why placeholders? SQL injection

Never build SQL by gluing strings:

```
name = input("Name: ")
con.execute("INSERT INTO students (name, year, program) VALUES ('" + name + "', 1, 'CS')")   #
DANGEROUS
```

If someone types `x', 1, 'CS'); DROP TABLE students;--`, the text becomes part of the SQL command. With `?`, the value is always treated as *data*, never as SQL. The test `test_d1_no_sql_injection` checks exactly this.

D.3 Exercises D1–D3 (practice/school_repo.py)

The function `students_in_program` is done as an example. Complete the other three:

#	Function	Hint
D1	`add_student(con, name, year, program) -> int`	`INSERT … VALUES (?, ?, ?)`, then `con.commit()`, return `cur.lastrowid`
D2	`average_grade(con, code) -> float or None`	`SELECT AVG(grade) … WHERE course_code = ?`; `fetchone()` gives a tuple; round to 2
D3	`transcript(con, student_id) -> list[(title, credits, grade)]`	a JOIN of `enrollments` and `courses`, ordered by title

```
(.venv) $ pytest -q test_school_repo.py
```

```
.....                                                                    [100%]
5 passed
```

NOTE

The tests create a fresh database **in memory** (`sqlite3.connect(":memory:")`) for each test, so your `school.db` file is never changed by tests.

Part E — The Assistant Remembers

Now we apply the same ideas to the assistant. Go to `lab05/starter/`.

E.1 What should we store — and what not?

The assistant talks with students. It would be easy to save everything — but storing personal data that no feature needs is a risk (leaks, misuse) and, in Vietnam, is limited by the Personal Data Protection Law. Our rule: **store only what a feature needs.**

Data	Store?	Why
student name, phone	**no**	no feature needs it
the full question text	**no**	may contain personal details
a code for the conversation (session)	yes, but **scrambled** (hashed)	lets us count sessions without knowing who
intent, topic, answered or not, time taken	yes	tells us what to improve
the thumbs-up/down and an optional comment	yes	the whole point of feedback

"Hashed" means we run the session id through a one-way function (SHA-256 with a secret *salt* from `.env`). The same session always gives the same code, but the code cannot be turned back into the id.

E.2 One class = one table (SQLModel)

Open `src/assistant_api/db.py`. We use **SQLModel**, a library that lets you describe a table as a Python class — you already know classes from OOP:

```
class Interaction(SQLModel, table=True):
    id: int | None = Field(default=None, primary_key=True)     # primary key, filled by the
database
    request_id: str = Field(index=True, unique=True)           # the id the API gives to the
UI
    session_id: str = Field(foreign_key="chat_session.id")     # foreign key -> chat_session
    intent: str                                                # "fee", "office", "rule", ...
    topic: str | None = None                                   # e.g. "H26-5.3"
    resolved: bool = True                                      # False = the bot did not know
    degraded: bool = False                                     # True = a backup answer was
used
    latency_ms: float = 0.0                                    # how long the answer took
    created_at: datetime = Field(default_factory=utcnow)
```

In the class	In the database
the class `Interaction`	the table `interaction`
an attribute `intent: str`	a column of type text
`primary_key=True`	the primary key
`foreign_key="chat_session.id"`	a foreign key

In the class	In the database
creating an object and saving it	`INSERT`

SQLModel writes the SQL for you. You can still read the equivalent SQL in `sql/migrations/001_init.sql`.

E.3 The repository: one place for all database code

`Repository` groups every database operation, so the API never contains SQL. Read the worked example `record_interaction`:

```
def record_interaction(self, raw_session, request_id, intent, topic, resolved, degraded,
latency_ms):
    sid = hash_session(raw_session, self.salt)           # scramble the session id
    with Session(self.engine) as db:                     # open a conversation with the
database
        if db.get(ChatSession, sid) is None:             # first message of this session?
            db.add(ChatSession(id=sid))
        db.add(Interaction(request_id=request_id, session_id=sid, intent=intent, topic=topic,
                           resolved=resolved, degraded=degraded, latency_ms=latency_ms))
        db.commit()                                      # save both — or neither
```

The pattern is always: **open a session** → **add or query** → **commit**. `commit()` saves everything together; if something fails before it, nothing is saved (this is called a **transaction**).

E.4 R1 — save feedback (4 lines)

```
def add_feedback(self, request_id: str, rating: int, comment: str | None) -> Feedback | None:
    with Session(self.engine) as db:
        exists = db.exec(select(Interaction).where(Interaction.request_id ==
request_id)).first()
        if exists is None:
            return None                       # the API turns this into 404
        fb = Feedback(request_id=request_id, rating=rating, comment=comment)
        db.add(fb)
        db.commit()
        db.refresh(fb)                        # load the id the database just gave it
        return fb
```

Why check `exists` first? Feedback must be about a real answer. Without the check, anyone could send feedback for an id that never existed.

E.5 R2 — list feedback, newest first (1 line)

In SQL you would write `SELECT * FROM feedback ORDER BY created_at DESC LIMIT 100`. In SQLModel:

```
return list(db.exec(select(Feedback).order_by(Feedback.created_at.desc()).limit(limit)))
```

E.6 Run it

```
(.venv) $ pytest -q
.........................                                [100%]
27 passed
(.venv) $ uvicorn assistant_api.main:create_app --factory --reload
```

In `/docs`: send a chat message, copy the `request_id` from the answer, send feedback with that id (201), then send feedback with the id `nope` (404). Stop and restart the server, list feedback with the API key — **it is still there**. Open `assistant.db` in DB Browser and look at the three tables.

Part F — Questions About the Assistant (Should)

Fill the database with **invented** data and ask real questions.

```
(.venv) $ python scripts/migrate.py assistant.db
apply 001_init.sql
apply 002_add_faq.sql
(.venv) $ python scripts/seed.py assistant.db 300
seeded 300 interactions into assistant.db
```

> **NOTE**
> A **migration** is a numbered SQL file that changes the database structure one step at a time. `migrate.py` remembers which files already ran, so running it twice is safe. You only need to *use* it today.

Open `assistant.db` in DB Browser and write three queries in `sql/my_queries.sql`:

#	Question	Hint
F1	How many questions per intent?	`GROUP BY intent`
F2	Average time taken per intent?	`AVG(latency_ms)`, `ROUND(..., 1)`
F3	Which topics got the most thumbs-down?	`JOIN feedback f ON f.request_id = i.request_id WHERE f.rating = -1`

Expected shape for F1 (numbers depend on the seed):

```
intent    requests
--------  --------
rule      140
fee       61
office    47
greeting  26
unknown   26
```

Homework (about 1 hour)

H1 — Finish the SQL practice (must)

Reach **15/15** in `python check.py`. If you are stuck on a join, write the Python loop version first (Part C.8), then translate it.

H2 — One question about the assistant (should)

Write one more useful question about the assistant data (for example: *"Which hour of the day is the busiest?"* or *"What share of answers used a backup path?"*), and the query that answers it. Paste the result.

H3 — Bonus: design review

Open `sql/bad_schema.sql` — a badly designed table. List three problems and how our design fixes them (worksheet `lab05/worksheets/data_model.md`).

How to Submit

What	When	How
`my_answers.sql` (score shown by `check.py`) + D1–D3	end of the session	PR link on the LMS
R1–R2, tests green	end of the session (draft PR OK)	PR link on the LMS
team merge, tag `v0.5-data`	23:59 the day before Lab 6	Tech Lead: tag link
H1 15/15, H2 query + result	23:59 the day before Lab 6	PR link on the LMS

Grading checklist

Criterion	Points
SQL Q1–Q10 correct	3
SQL Q11–Q15 correct	2
D1–D3 tests pass (incl. the injection test)	2
R1–R2 tests pass; feedback survives a restart	2
F1–F3 or H2 query with result	1
Bonus design review	+1
Total	**10 (+1)**

Troubleshooting

Problem	What to do
`school.db not found`	run `python make_school_db.py` in `lab05/practice/`
`no such table`	wrong database file open, or you did not run `make_school_db.py` / `migrate.py`
`no such column: Name`	column names are case-sensitive in some tools; check spelling
Text comparison finds nothing	use single quotes `'CS'`, and check upper/lower case
`NULL = NULL` finds nothing	use `IS NULL`
`database is locked`	DB Browser has unsaved changes — click *Write Changes* or close it
Changes disappear	you forgot `con.commit()` (Python) or *Write Changes* (DB Browser)
`IntegrityError: UNIQUE constraint failed`	the same id was inserted twice
`sqlite3` command not found (Windows)	you do not need it — use DB Browser

Glossary

Word	Meaning
database	a program that stores data on disk and answers questions about it
table / row / column	a set of records / one record / one field of every record
primary key	a column whose value is unique for every row
foreign key	a column holding the primary key of a row in another table
NULL	"no value" — like `None`
SQL	the language used to ask a database for data
query	one SQL statement, usually a `SELECT`
JOIN	combine rows of two tables that match on a key
transaction / commit	a group of changes saved together / the moment they are saved
SQL injection	an attack where user input is executed as SQL — prevented by `?` placeholders
hash	a one-way scramble: same input → same code, but the code cannot be reversed
migration	a numbered file that changes the database structure step by step

Self-Check

1. What is the difference between a primary key and a foreign key?
2. Write a query for the names of year-2 students, A to Z.
3. What does `GROUP BY` do, in one sentence?
4. Why do we use `?` placeholders in Python?
5. Why does the assistant not store the full question text?

Answers. 1) A primary key identifies a row in its own table; a foreign key stores the primary key of a row in another table. 2) `SELECT name FROM students WHERE year = 2 ORDER BY name;` 3) It makes one group per distinct value and computes COUNT/AVG/SUM for each group. 4) So user input is always treated as data, never as SQL — preventing SQL injection. 5) It may contain personal information, and no feature needs it.

Resources

- SQLBolt — interactive lessons 1–12 (sqlbolt.com)
- DB Browser for SQLite — sqlitebrowser.org
- Python docs — `sqlite3` tutorial
- SQLModel — *Tutorial: Create a Table*, *Create Rows*, *Read Data* (sqlmodel.tiangolo.com)